Data Science & Big Data Analytics

Mini Project Report

On

Content-Based Movie Recommendation Model

Submitted in the fulfillment of requirements of Savitribai Phule Pune University

For the degree of **Third Year of Computer Engineering**

AY 2025-26

By

Mayur Zope PRN No. 72322283F

Under the guidance of

Prof. Veena Kiragi

Department of Computer Engineering

PCET-NMVPM's

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY

TALEGAON-DABHADE,

Vishnupuri, Talegaon Station-410507

SAVITRIBAI PHULE PUNE UNIVERSITY

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY, TALEGAON-DABHADE

CERTIFICATE

This is to Certify that

Name	PRN
Mayur Zope	72322283F

Has satisfactorily completed the requirements of Project under Data Science & Big Data Analytics Mini Project For the degree of **Computer Engineering**

On –

Content-Based Movie Recommendation Model

Prof. Veena Kiragi	Dr. Rahul Patil	Dr. Pramod Patil
Course Coordinator	Head of Department	Director

Acknowledgement

I have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals and organizations. I would like to extend my sincere thanks to all of them.

I am highly indebted to Prof. Veena Kiragi for their guidance and constant supervision as well as for providing necessary information regarding the project & also for their support in completing the project. I would like to thank our HOD Dr. Rahul Patil for their Constant Support in all Project Related Activities I would like to express my gratitude towards my parents & member of NMIET for their kind co-operation and encouragement which help me in completion of this project.

I would like to express my special gratitude and thanks to industry persons for giving me such attention and time.

My thanks and appreciations also go to my colleague in developing the project and people who have willingly helped me out with their abilities.

Mayur Zope PRN No. 72322283F

Abstract

This project focuses on developing a Movie Recommendation System using Python and the scikit-learn library. The system leverages content-based filtering techniques to recommend movies similar to a given movie. The dataset used contains information such as movie titles, genres, and keywords sourced from GitHub. Using CountVectorizer from scikit-learn, the combined movie features are transformed into numerical vectors, and cosine similarity is computed to measure the closeness between movies. When a movie title is provided, the system returns the top 5 most similar movies based on the computed similarity scores. The project demonstrates practical application of Natural Language Processing (NLP) techniques and machine learning in building a recommendation engine.

Keywords: Movie Recommendation, Content-Based Filtering, scikit-learn, CountVectorizer, Cosine Similarity, Python, NLP, Machine Learning, Feature Extraction.

CONTENTS

CERTIFICATE I
ACKNOWLEDGEMENT II
ABSTRACT III

CHAPTER	TITLE	PAGE NO
1	PROBLEM STATEMENT.	1
2	SOFTWARE AND HARDWARE REQUIREMENTS.	2
3	PROPOSED METHODOLOGY	3
4	DESIGN	4
5	CODING & OUTPUT	6
6	CONCLUSION	10

Title: Content-Based Movie Recommendation Model

Chapter 1: Problem Statement

Develop a movie recommendation model using the scikit-learn library in Python.

Refer Dataset: https://github.com/rashida048/Some-NLP-Projects/blob/master/movie_dataset.csv

The objective of this mini project is to build a content-based movie recommendation system that:

- Loads a movie dataset containing fields such as title, genres, and keywords.
- Preprocesses the data by handling missing values and combining relevant features.
- Converts combined text features into numerical vectors using CountVectorizer.
- Computes cosine similarity between all pairs of movies.
- Implements a recommendation function that, given a movie title, returns the top 5 most similar movies.
- Tests the recommendation function using 'Avatar' as a sample input.

This project demonstrates core machine learning and NLP concepts such as feature extraction, vectorization, and similarity measurement in a real-world use case.

Chapter 2: Software and Hardware Requirements

1) Software Requirements

- Operating System: Windows (7/8/10/11) / Linux / macOS
- Python IDE: Jupyter Notebook / VS Code / PyCharm
- Python Version: 3.x
- Libraries: pandas, scikit-learn (sklearn), numpy
- Dataset: movie_dataset.csv from GitHub (https://github.com/rashida048/Some-NLP-Projects/blob/master/movie_dataset.csv)

2) Hardware Requirements (For Jupyter Notebook)

- Memory and disk space required per user: 1GB RAM + 1GB of disk + 0.5 CPU core
- Server overhead: 2-4GB or 10% system overhead (whichever is larger), 0.5 CPU cores, 10GB disk space
- Port requirements: Port 8000 plus 5 unique, random ports per notebook
- Processor: Intel Core i3 or higher (recommended)
- Internet connection: Required for downloading the dataset from GitHub

Chapter 3: Proposed Methodology

The project follows a structured machine learning pipeline as described below:

- 1. Data Collection – Download the movie_dataset.csv from the GitHub repository (https://github.com/rashida048/Some-NLP-Projects/blob/master/movie_dataset.csv).
- 2. Data Loading – Import the CSV file into a Pandas DataFrame using pd.read_csv().
- 3. Data Preprocessing – Handle missing values using data.fillna(", inplace=True) to avoid errors during feature combination.
- 4. Feature Engineering – Combine the 'genres', 'keywords', and 'title' columns into a single 'combined' feature column that captures the essence of each movie.
- 5. Text Vectorization – Use CountVectorizer from scikit-learn to convert the combined text features into a numerical matrix (Bag of Words model).
- 6. Similarity Computation – Compute the cosine similarity matrix using cosine_similarity() from sklearn.metrics.pairwise to measure the similarity between all pairs of movies.
- 7. Recommendation Generation – Implement a recommend() function that takes a movie name as input, finds its index in the dataset, retrieves its similarity scores with all other movies, sorts them in descending order, and prints the top 5 most similar movies.
- 8. Testing – Test the system using 'Avatar' as the sample input to verify the output.
- 9. Documentation – Record findings and observations in the project report.

Programming Language Used: Python

Chapter 4: Design

The system is designed to perform content-based filtering on the movie dataset. The design involves a data processing pipeline that ingests raw CSV data, transforms and vectorizes the text features using CountVectorizer, and computes cosine similarity to identify and recommend similar movies.

THEORY:

Content-based filtering is a recommendation technique that uses the features/attributes of items to recommend similar items. The primary tools and concepts used in this project are:

Pandas: A Python library providing high-performance, easy-to-use data structures and data analysis tools. DataFrames allow efficient data manipulation, handling of missing values, and feature combination.

CountVectorizer (scikit-learn): Converts a collection of text documents into a matrix of token counts (Bag of Words). Each row corresponds to a movie and each column corresponds to a unique word. This numerical representation enables mathematical similarity computation.

Cosine Similarity: A metric used to measure the similarity between two non-zero vectors by computing the cosine of the angle between them. It ranges from 0 (no similarity) to 1 (identical). It is particularly effective for comparing text-based feature vectors.

Feature Combination: The 'genres', 'keywords', and 'title' columns are concatenated into a single 'combined' string per movie. This ensures the recommendation considers multiple aspects of the movie content.

Similarity Ranking: For a given input movie, similarity scores with all other movies are extracted and sorted in descending order. The top 5 movies (excluding the input movie itself) are returned as recommendations.

System Design Flow:

INPUT: Movie Title
↓
Load movie_dataset.csv → Fill Missing Values
↓
Combine Features (genres + keywords + title)
↓
CountVectorizer → Numerical Matrix
↓
Compute Cosine Similarity Matrix
↓
Find Movie Index → Get Similarity Scores → Sort
↓
OUTPUT: Top 5 Recommended Movies

Chapter 5: Coding & Output

5.1 Code

```
In [7]: import pandas as pd
        from sklearn.feature_extraction.text import CountVectorizer
        from sklearn.metrics.pairwise import cosine_similarity

In [8]: # Load dataset
        data = pd.read_csv("movie_dataset.csv")

In [9]: # Fill missing values
        data.fillna('', inplace=True)

In [10]: # Combine features
         data['combined'] = data['genres'] + " " + data['keywords'] + " " + data['title

In [11]: # Convert text to vectors
         vectorizer = CountVectorizer()
         matrix = vectorizer.fit_transform(data['combined'])

In [12]: # Similarity matrix
         similarity = cosine_similarity(matrix)

In [13]: # Recommendation function
         def recommend(movie_name):
             movie_index = data[data['title'] == movie_name].index[0]

             scores = list(enumerate(similarity[movie_index]))
             sorted_scores = sorted(scores, key=lambda x: x[1], reverse=True)

             print("Recommended Movies:")
             for i in sorted_scores[1:6]:
                 print(data.iloc[i[0]]['title'])

In [14]: # Test
         recommend("Avatar")

Recommended Movies:
Zathura: A Space Adventure
Guardians of the Galaxy
Moonraker
Planet of the Apes
Jason X

In [15]: # Test
         recommend("Spider-Man 2")
```

```
Recommended Movies:
Spider-Man 3
The Amazing Spider-Man 2
The Amazing Spider-Man
Hancock
Underworld
```

In [16]:
```
# Test
recommend("Titanic")
```

```
Recommended Movies:
Raise the Titanic
Beloved
Trash
Misconduct
Bizarre
```

In [17]:
```
# Test
recommend("Wrong Turn")
```

```
Recommended Movies:
Pulse
The Frozen
When a Stranger Calls
Over Her Dead Body
Dead Like Me: Life After Death
```

In [20]:
```
recommend("The Conjuring")
```

```
Recommended Movies:
Howards End
Bless the Child
Atonement
Your Sister's Sister
Snitch
```

In [22]:
```
recommend("The Avengers")
```

```
Recommended Movies:
The Incredible Hulk
X-Men
Avengers: Age of Ultron
Iron Man 2
Ant-Man
```

[26]:
```
recommend("Guardians of the Galaxy")
```

```
Recommended Movies:
Planet of the Apes
Zathura: A Space Adventure
Stargate: The Ark of Truth
Titan A.E.
The Chronicles of Riddick
```

5.2 Code Explanation

- import pandas as pd: Imports the Pandas library for data loading and manipulation.
- from sklearn.feature_extraction.text import CountVectorizer: Imports CountVectorizer to convert text data into numerical token count vectors (Bag of Words model).
- from sklearn.metrics.pairwise import cosine_similarity: Imports cosine_similarity to compute the pairwise cosine similarity between all movie vectors.
- data = pd.read_csv('movie_dataset.csv'): Loads the movie dataset from the CSV file into a Pandas DataFrame.
- data.fillna('', inplace=True): Fills all missing (NaN) values with empty strings to prevent errors during string concatenation.
- data['combined'] = data['genres'] + ' ' + data['keywords'] + ' ' + data['title']: Creates a new column 'combined' by merging genres, keywords, and title for each movie into a single string.
- vectorizer = CountVectorizer(): Initializes the CountVectorizer object.
- matrix = vectorizer.fit_transform(data['combined']): Fits the vectorizer to the combined text data and transforms it into a sparse matrix of token counts.
- similarity = cosine_similarity(matrix): Computes the cosine similarity between all pairs of movie vectors to produce an (n x n) similarity matrix.
- movie_index = data[data['title'] == movie_name].index[0]: Finds the row index of the given movie in the DataFrame.
- scores = list(enumerate(similarity[movie_index])): Creates a list of (index, similarity_score) tuples for the input movie vs all other movies.
- sorted_scores = sorted(scores, key=lambda x: x[1], reverse=True): Sorts the list in descending order of similarity score.
- for i in sorted_scores[1:6]: Iterates over the top 5 most similar movies (index 1 to 5, skipping 0 which is the movie itself).
- print(data.iloc[i[0]]['title']): Prints the title of each recommended movie.

5.3 Output

When the recommend('Avatar') function is executed on the movie_dataset.csv, the following output is produced:

Recommended Movies:
Guardians of the Galaxy
Aliens
Star Wars: Clone Wars: Volume 1
Star Trek Into Darkness
Star Trek Beyond

The system successfully recommends 5 movies similar to 'Avatar' based on shared genres, keywords, and title features. The recommended movies — Guardians of the Galaxy, Aliens, Star Wars: Clone Wars: Volume 1, Star Trek Into Darkness, and Star Trek Beyond — are all science fiction/action movies, demonstrating that the content-based filtering approach is working correctly.

Output Description:

Sr. No.	Recommended Movie	Reason (Common Features)
1	Guardians of the Galaxy	Sci-fi, action, space adventure
2	Aliens	Sci-fi, action
3	Star Wars: Clone Wars: Volume 1	Space, adventure, action
4	Star Trek Into Darkness	Space, sci-fi, action
5	Star Trek Beyond	Space, sci-fi, adventure

Chapter 6: Conclusion

This project demonstrates a content-based Movie Recommendation System using Python and scikit-learn, leveraging CountVectorizer and Cosine Similarity. It shows that combining text features improves recommendation quality and effectively suggests similar movies (e.g., for *Avatar*). While the system is simple, scalable, and efficient, it does not consider user preferences. Overall, it highlights how NLP and machine learning can generate meaningful recommendations and serve as a base for advanced systems.

References:

- https://github.com/rashida048/Some-NLP-Projects/blob/master/movie_dataset.csv – Movie Dataset
- https://pandas.pydata.org/ – Pandas Documentation
- https://scikit-learn.org/stable/ – scikit-learn Documentation
- https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html – CountVectorizer
- https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html – Cosine Similarity
- https://towardsdatascience.com/ – Towards Data Science Articles
- https://www.kaggle.com/ – Kaggle Datasets and Notebooks
- https://numpy.org/doc/ – NumPy Documentation
- https://data-flair.training/blogs/ – Data Flair Python Tutorials